

Model for Software Testing

A field guide to the tester's mindset, the models we choose, and the real cost of skipping the work.

UNIVERSITY QUESTION · MU — MAY 17, MAY 19

“What is software testing? Describe software testing model with a neat diagram.”

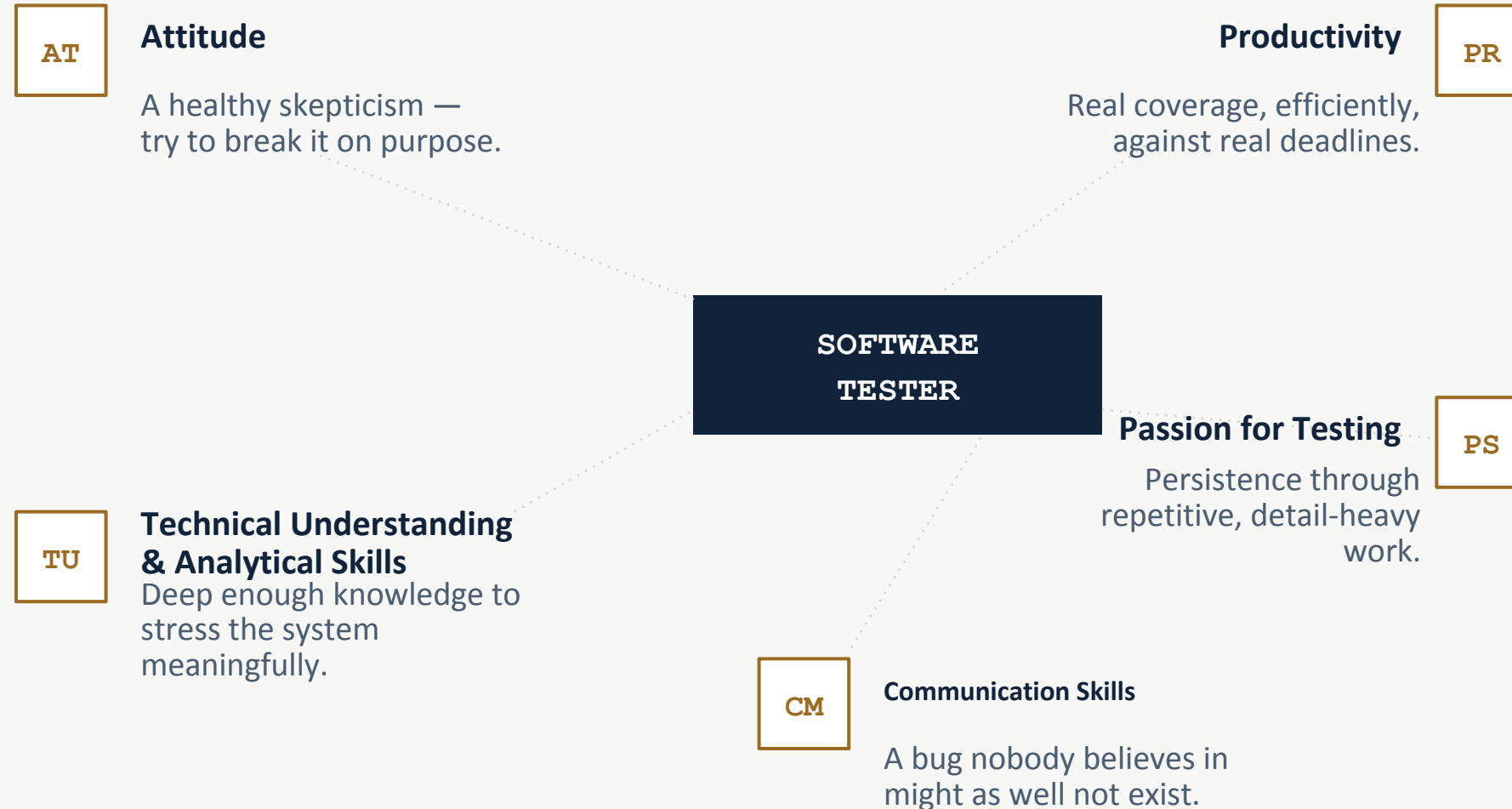
“A majority of software development time is now spent on testing — not writing new code.”

— Software Testing & Quality Assurance, on where the modern development clock actually goes

Software testing is now a significant, integral part of software development. Ideally, it is introduced in every phase of the development life cycle — not saved for the end, and not treated as a formality before shipping.

The Software Testing Model

Five qualities that define a tester's practice — reconstructed as a technical schematic.



Choosing the Right Approach

Software development models are the various processes or methodologies selected for a project, depending on that project's own aims and goals. Many development life-cycle models exist, each achieving different objectives — each with its own advantages and disadvantages.

Software testing is an integral part of the software development life cycle. You must choose a particular model depending on the project's deliverables and the complexity of the project — there is no universally “best” choice.

DEFINITION

Model-Based Testing

A software testing technique where the run-time behaviour of software under test is checked against predictions made by a model.

A model is a description of a system's behaviour — expressed in terms of:

- Input sequences
- Actions
- Conditions
- Output
- Flow of data, input to output

The Boeing 737 MAX

What happens when a single untested assumption becomes a matter of life and death.

Boeing's 737 has flown since 1967 — one of the most trusted aircraft families in aviation history. To compete with a rival's newer, more fuel-efficient jet, Boeing fitted the 737 MAX with larger, repositioned engines. Those engines changed how the aircraft handled at certain angles, so engineers wrote a software fix: MCAS, the Maneuvering Characteristics Augmentation System — designed to automatically push the aircraft's nose down if it sensed a dangerously high angle of attack.

THE DESIGN DECISION AT THE CENTRE OF THIS CASE

MCAS made that automatic, forceful decision based on a reading from just one sensor — with no cross-check against a second, independent source.

How It Unfolded



346

lives lost, combined, across both flights

EST. COST TO BOEING

\$20B+

FLEET GROUNDED

~20 months

ROOT CAUSE

A single point of failure, trusted completely

MCAS made a life-critical, automatic decision based on input from a single angle-of-attack sensor, with no cross-check against a second, independent source. When that sensor failed on two separate flights, the system repeatedly forced the nose down — and pilots who had not been fully briefed on MCAS's existence could not recover in time.

Investigators later found that some of Boeing's own engineers and test pilots had raised concerns about this exact design before the crashes.

Transferable Lessons

LESSON 01

Redundancy is not optional

Any system taking an irreversible, safety-critical action needs more than one independent source of truth.

LESSON 02

Communicate risk upward, clearly

A concern raised but not escalated effectively is functionally the same as a concern never raised at all.

LESSON 03

Users must know a system exists

You can't expect anyone to defend against a failure mode they were never told was possible.

LESSON 04

Cutting corners is rarely cheaper

The cost of the crisis dwarfed whatever time or money was saved by skipping rigorous testing beforehand.

Back to Fundamentals

“ *At the end, software is developed by a team of human developers, all with different viewpoints and approaches. Even the smartest person has the tendency to make an error. It is not possible to create software with zero defects without incorporating software testing into the development cycle.*

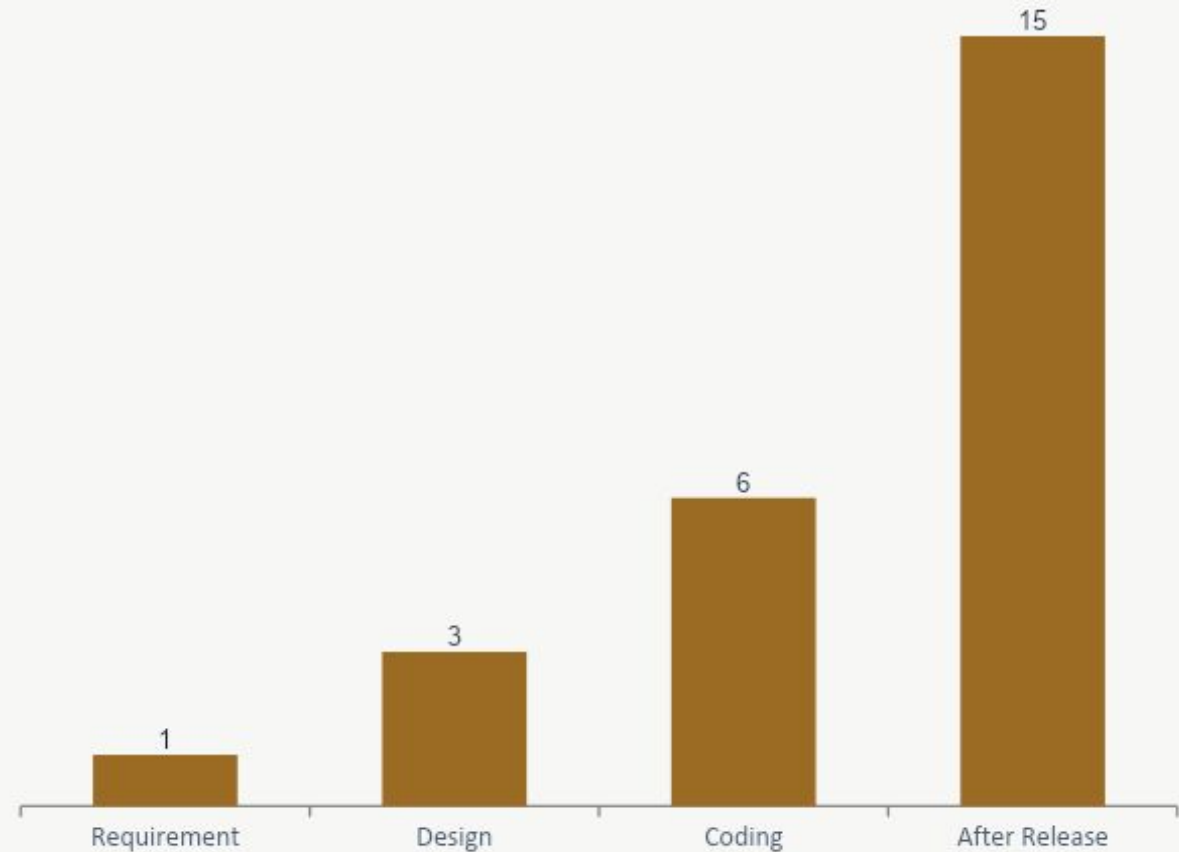
No matter how well a design looks on paper, once development starts and testing begins, you will definitely find defects. You cannot achieve software quality without software testing.

Even when testers aren't involved in actual coding, they should work closely with developers — for best results, testing and coding should go hand in hand.

Not Every Defect Is a Threat — But Every Delay Has a Cost

Every software application has some defects embedded in it, but not every defect is a threat to the system. Testing helps reduce the overall cost of a project: skip it early to save a small amount of money, and it becomes far more expensive later — tracing a defect back gets harder as development progresses, and fixing one can introduce another in a different module.

Relative Cost to Fix a Defect, by Stage Found



The Sign-Off Sheet

What a disciplined testing effort makes possible, in the client's own terms.

- ☐ **No argument on requirement variation**
When software is delivered, there are no arguments about variation from the original requirements testing is what earns a company the right to make that claim honestly.
- ☐ **Reputation follows quality**
Well-tested software is good-quality software, and good quality means better client feedback and reviews directly strengthening a company's market reputation.
- ☐ **A formal test plan, before execution**
All testing activities require planning. A test plan outlines in detail how each activity is carried out covering every aspect thoroughly, with no repetition, so time and effort aren't wasted.
- ☐ **Test cases written in parallel to specs**
The latest trend is to involve testers directly in writing specifications. Writing test cases in parallel forces critical thinking about requirements revealing what's untestable while it's still cheap to fix.

Key Takeaways

- 01 Fig. 1.2.6: five human qualities — attitude, technical skill, productivity, passion, communication — define a great tester.
- 02 Choose development & testing models by project goals, deliverables, and complexity — no single model fits all.
- 03 Boeing 737 MAX: 346 lives lost because a life-critical decision relied on one sensor, with no redundancy.
- 04 Not every defect is a threat — but skipping early testing to save money almost always costs far more later.
- 05 Plan testing with a formal test plan, and write test cases in parallel with specifications, not after.